

Как устроена авторизация, двухфакторка и смена пароля

Пошаговый визуальный разбор: что видит пользователь на экране, какой именно запрос в этот момент улетает на сервер (реальная панель Network из Chrome DevTools), и что происходит внутри бэкенда — какие проверки, какой код и какая логика отрабатывают за эти доли секунды. Документ рассчитан на читателя без опыта в разработке.

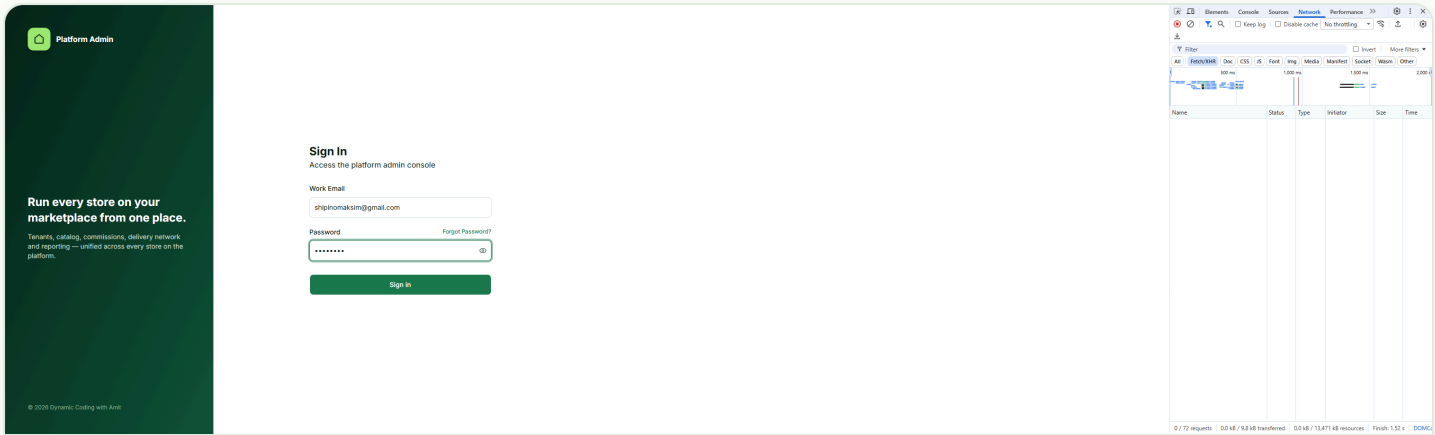
-  Слева на скриншотах — то, что видит пользователь в браузере
-  Справа — настоящая панель Network из Chrome DevTools в этот же момент
-  Тёмные блоки — реальный код бэкенда, с подсветкой

- | | |
|--|---|
| 01 Вход в аккаунт, когда включена 2FA | 02 Отключение двухфакторной аутентификации |
| 03 Обычный вход (2FA выключена) | 04 Повторное включение 2FA |
| 05 Смена пароля | 06 Редактирование профиля |
| 07 Что защищает аккаунт: итоги | |

1 Вход в аккаунт, когда включена 2FA

У аккаунта сейчас включена двухфакторная аутентификация (2FA). Это значит, что одного пароля недостаточно — после него сервер потребует ещё одноразовый код из письма. Смотрим, как это выглядит и что происходит на сервере на каждом шаге.

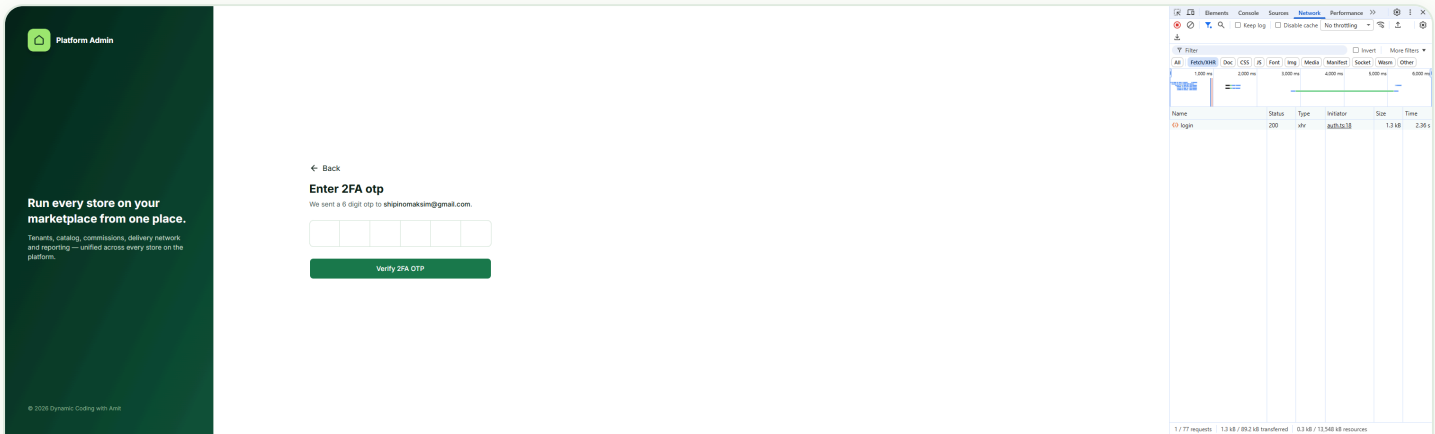
Шаг 1 — заполняем форму входа FRONTEND



Форма входа заполнена, кнопка «Sign in» ещё не нажата. Справа в Network пока пусто — ничего не отправлено.

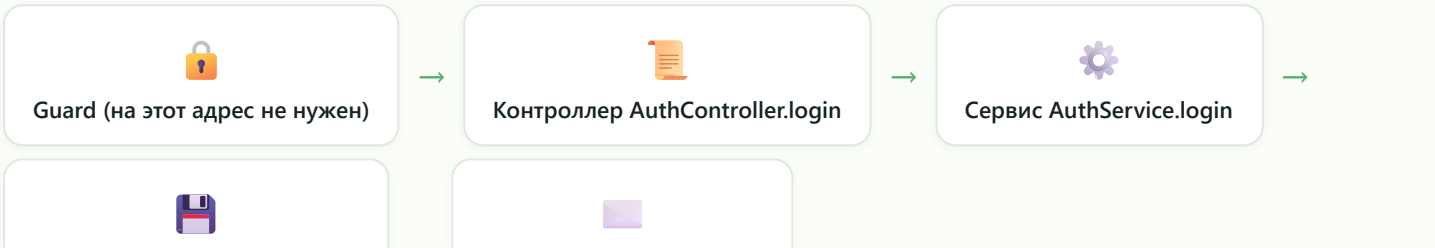
Пока вы печатаете email и пароль, в сеть ничего не уходит — проверяется только на глаз, что поля не пустые. Реальная проверка происходит только после нажатия «Sign in».

Шаг 2 — сервер просит код FRONTEND BACKEND



Запрос login (200) уже прошёл, но вместо ключей доступа сервер вернул «нужен код 2FA» — браузер сам переключил на страницу ввода кода.

Это ключевой момент: запрос `login` вернул **200 OK** (пароль верный!), но без токенов доступа. Вместо них сервер прислал короткоживущий «билет на второй шаг». Разберём, что произошло внутри:



Простыми словами. Вход не защищён "охранником" (Guard'ом), потому что на этом шаге пользователь ещё не залогинен — охранять нечего, наоборот, сюда должен суметь достучаться кто угодно с логином и паролем. А вот дальше контроллер (это как диспетчер: принял запрос, передал по адресу и вернул ответ) отдаёт всю работу сервису. Сервис:

- 1 Находит пользователя по email в базе данных.
- 2 Сверяет пароль с тем, что хранится в базе (в базе лежит не сам пароль, а его необратимый «отпечаток» — хэш; сравнение идёт по отпечаткам).
- 3 Видит, что у аккаунта включена 2FA — и вместо выдачи токенов генерирует 6-значный код, сохраняет его (тоже в виде хэша, не в открытом виде!) и отправляет на почту через сервис Resend.
- 4 Подписывает специальный короткоживущий токен (живёт 15 минут) — это тот самый «билет на второй шаг», который видно в ответе на скриншоте как `twoFactorToken`.

```
// backend/src/auth/auth.service.ts
async login(email: string, password: string, context = {}) {
  const user = await this.usersService.findByEmail(email);

  // Сравнение с "фиктивным" хэшем, даже если пользователь не найден –
  // чтобы ответ занимал одинаковое время в обоих случаях (защита от
  // подбора существующих email по разнице в скорости ответа).
  const passwordMatches = await bcrypt.compare(
    password,
    user?.passwordHash ?? ABSENT_USER_PASSWORD_HASH,
  );

  if (!user || !passwordMatches) {
    throw new UnauthorizedException('Invalid email or password');
  }

  if (user.twoFactorEnabled) {
    // Не выдаём токены сразу – создаём одноразовый код и билет
    // на второй шаг проверки.
    const result = await this.twoFactorCreateOtpToken(user.id);
    return {
      twoFactorRequired: true,
      twoFactorToken: result.twoFactorToken,
      message: result.message,
    };
  }

  const tokens = await this.issueTokenPair(user, context);
```

```
return { ... tokens, userType: user.userType };  
}
```

Почему пароль сравнивают даже для несуществующего email?

Это защита от «угадывания» зарегистрированных адресов. Если бы для несуществующего email сервер отвечал мгновенно, а для существующего - с небольшой задержкой (на сравнение пароля), по одной только скорости ответа можно было бы понять, зарегистрирован ли email в системе. Поэтому сравнение хэшей выполняется всегда, даже вхолостую.

Шаг 3 — вводим код из письма FRONTEND

Platform Admin

Run every store on your marketplace from one place.

Tenants, catalog, commissions, delivery network and reporting — unified across every store on the platform.

© 2020 Dynamic Coding with Java

← Back

Enter 2FA otp

We sent a 6 digit otp to shipinomekaim@gmail.com.

4

8

6

8

7

5

Verify 2FA OTP

Elements

Console

Source

Network

Performance

Y Filter

XHR

logn

200

xhr

auth.js:18

1.3 kB

2.36 s

Шестизначный код, только что пришедший на почту, введён в поле.

Шаг 4 — код проверен, доступ открыт FRONTEND BACKEND

Platform

DashboardPage

Dashboard

Sellers

Commission & Payouts

Global Catalog

Delivery Network

Customers

Orders

Supports

Marketing

Reports

Users & Permissions

Settings

Shipino Mekaim

shipinomekaim@gmail.com

logn

200

xhr

auth.js:18

1.3 kB

2.36 s

2fa-verify-login-otp

200

xhr

auth.js:52

1.7 kB

723 ms

me

200

xhr

auth.js:22

1.4 kB

450 ms

me

204

xhr

auth.js:22

1.2 kB

888 ms

После кнопки «Verify»: запрос 2fa-verify-login-otp (200) вернул уже настоящие токены; следом два запроса me - профиль подгружается на дашборд.



Контроллер 2fa-verify-login-otp



Сервис: проверка кода



Лимит попыток (5 раз)



Выдача токенов и сессии

Простыми словами. Сервер достаёт из базы последний код, который предназначен именно «для входа» (а не, например, для включения 2FA — у каждого кода есть метка «назначения», чтобы код от одного действия нельзя было подсунуть в другое). Сверяет введённое с хэшем. Если код неверный — увеличивает счётчик неудачных попыток; после пятой неудачной попытки код перестаёт приниматься вообще, даже если ввести его правильно — это защита от перебора всех 900 000 возможных комбинаций.

```
// backend/src/auth/auth.service.ts
async twoFactorVerifyLoginOtp(twoFactorToken: string, otp: string, context = {}) {
  // 1. Билет на второй шаг должен быть подлинным и ещё не истёкшим.
  let decode: TwoFactorTokenPayload;
  try {
    decode = await this.jwtService.verifyAsync(twoFactorToken, {
      secret: this.config.getOrThrow<string>('JWT_2FA_SECRET'),
    });
  } catch {
    throw new BadRequestException('Session is Expired');
  }

  const user = await this.prisma.user.findUnique({ where: { id: decode.userId } });
  if (user.status !== UserStatus.ACTIVE) {
    throw new ForbiddenException('This account is not active');
  }

  // 2. Проверка самого кода: назначение "LOGIN", лимит попыток, хэш.
  await this.verifyOtp(user.id, TwoFactorOtpPurpose.LOGIN, otp);

  // 3. Всё сошлось – только теперь выдаём настоящие токены доступа.
  const token = await this.issueTokenPair(user, context);
  return { ...token, userType: user.userType };
}
```

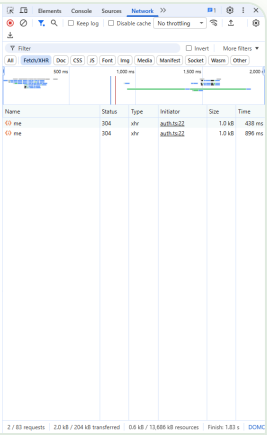
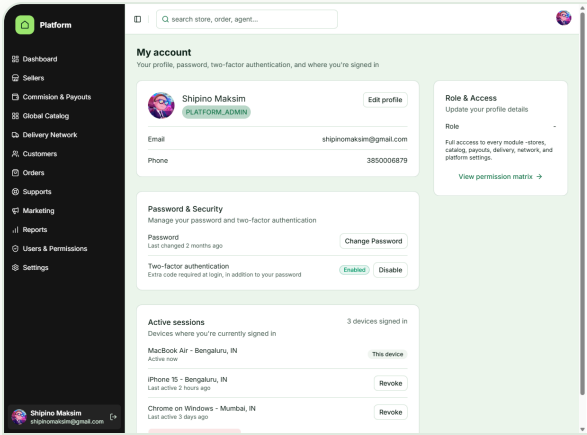
А что за два запроса "me" подряд?

Первый - GET /auth/me сразу после входа: страница дашборда запрашивает профиль, чтобы показать имя и аватар. Второй запрос с кодом 304 - это React в строгом режиме разработки (Strict Mode) намеренно повторяет эффект, чтобы разработчики ловили побочные эффекты; сервер отвечает "содержимое не изменилось", лишней нагрузки это не создаёт.

2 Отключение двухфакторной аутентификации

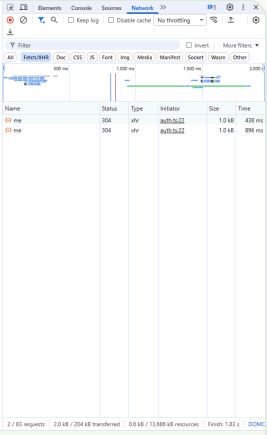
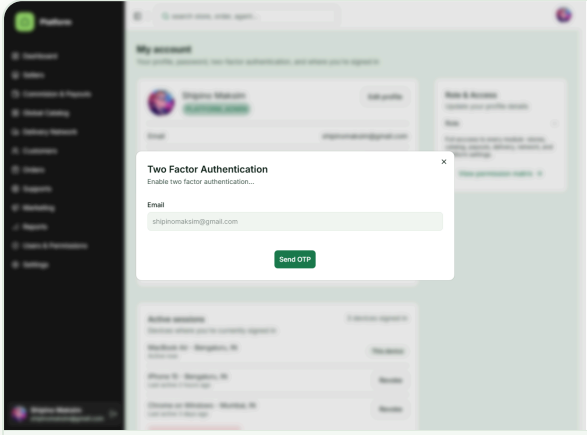
Раздел «Мой аккаунт» → «Password & Security». Включение и выключение 2FA технически — один и тот же переключатель на сервере: он всегда ставит значение, противоположное текущему. Поэтому что «включить», что «выключить» проходит через один и тот же код, только с разным начальным состоянием.

Шаг 1 — страница аккаунта FRONTEND



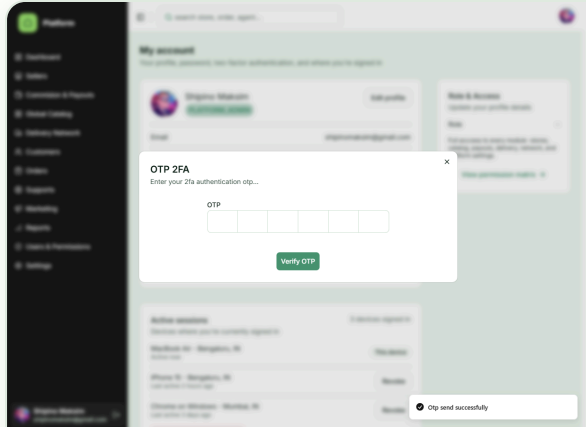
2FA сейчас включена (зелёный бейдж «Enabled»), поэтому кнопка называется «Disable».

Шаг 2 — открываем диалог FRONTEND



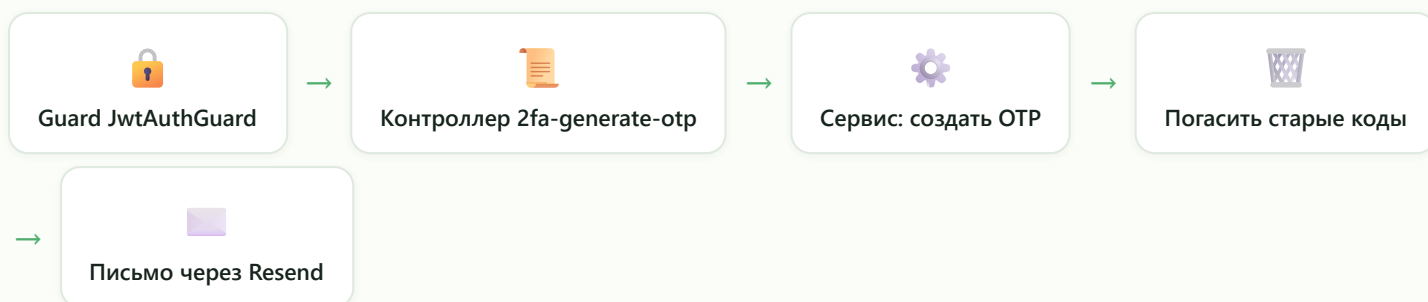
Диалог из двух шагов: сначала подтверждаем e-mail и просим код.

Шаг 3 — код отправлен FRONTEND BACKEND



Name	Status	Type	Initiator	Size	Time
me	200	xhr	auth.js:22	1.0 kB	438 ms
me	200	xhr	auth.js:22	1.0 kB	890 ms
2fa-generate-otp	200	xhr	auth.js:65	1.1 kB	1.1 s

Запрос 2fa-generate-otp (200) - на сервере создан новый код; форма переключилась на ввод шести цифр.



Простыми словами про Guard. В отличие от входа, этот запрос требует, чтобы человек уже был залогинен — поэтому перед контроллером стоит «охранник» `JwtAuthGuard`. Он смотрит на заголовок `Authorization: Bearer ...`, присланный браузером, проверяет подпись токена и, если всё в порядке, сверяется с базой — жива ли ещё сессия (её могли принудительно отозвать, например, при выходе с другого устройства). Если что-то не так — запрос отклоняется ошибкой 401 ещё до того, как выполнится хоть строчка бизнес-логики.

```
TYPESCRIPT

// backend/src/common/guards/jwt-auth.guard.ts
@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') {}

// backend/src/auth/strategies/jwt.strategy.ts
async validate(payload: JwtAccessPayload): Promise<AuthenticatedUser> {
  const session = await this.prisma.userSession.findUnique({
    where: { id: payload.sid },
    include: { user: true },
  });

  // Сессию могли отозвать вручную (например, кнопкой "выйти со всех устройств") - тогда токен всё ещё "живой" по подписи, но сервер всё равно откажет.
  if (!session || session.revokedAt || session.expiresAt <= new Date()) {
    throw new UnauthorizedException('Session is no longer valid');
  }

  if (session.user.status !== UserStatus.ACTIVE) {
    throw new UnauthorizedException('This account is not active');
  }
}
```

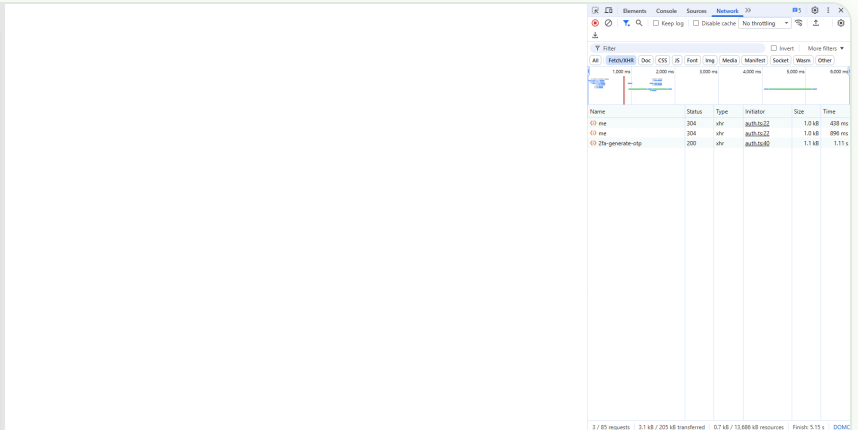
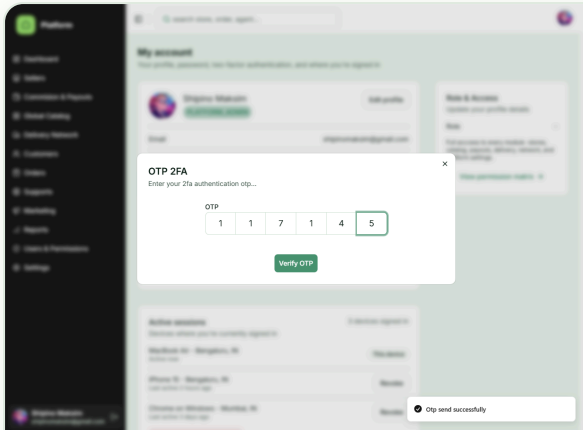
```

    }

    return {
      userId: session.userId,
      email: session.user.email,
      userType: session.user.userType,
      sessionId: session.id
    };
  }
}

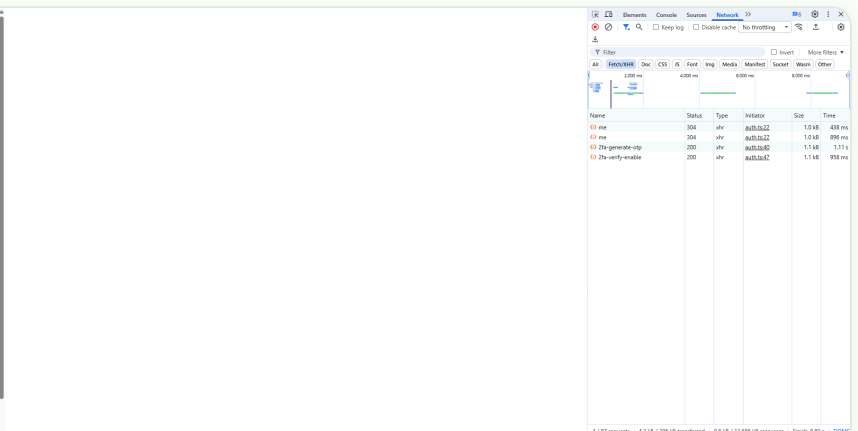
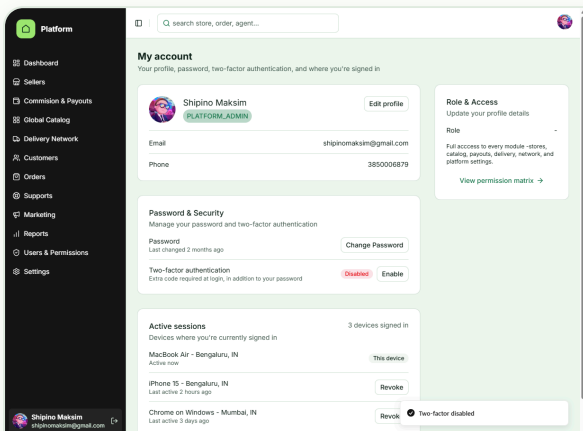
```

Шаг 4 — вводим код **FRONTEND**



Код из письма введён, готовы нажать «Verify OTP».

Шаг 5 — 2FA выключена **FRONTEND** **BACKEND**



Запрос 2fa-verify-enable (200) - на карточке появляется актуальное состояние переключателя.

TYPESCRIPT

```

// backend/src/auth/auth.service.ts
async twoFactorVerifyEnable(userId: string, otp: string) {
  const user = await this.prisma.user.findUnique({ where: { id: userId } });

  // Код обязательно должен быть с меткой ENABLE_TOGGLE - код, присланный
  // для входа, здесь не примут, даже если он ещё действителен.
  await this.verifyOtp(userId, TwoFactorOtpPurpose.ENABLE_TOGGLE, otp);

  const updatedUser = await this.prisma.user.update({
    where: { id: user.id },

```

```
    data: { twoFactorEnabled: !user.twoFactorEnabled }, // переключатель
  });

  return {
    success: true,
    message: `Two-factor ${updatedUser.twoFactorEnabled ? 'enabled' : 'disabled'}`,
    data: { twoFactorEnabled: updatedUser.twoFactorEnabled },
  };
}
```

Зачем коду "метка назначения"?

Коды для входа и коды для включения/выключения 2FA генерируются одной и той же функцией и хранятся в одной таблице. Без метки назначения (purpose) код, присланный для входа, можно было бы подсунуть в форму отключения 2FA - и наоборот. Метка гарантирует, что код, полученный для одной цели, не сработает для другой.

3 Обычный вход (2FA выключена)

Сейчас 2FA временно выключена (предыдущий раздел). Для сравнения — вот как выглядит вход без второго шага: пароль подошёл — и сразу выданы токены доступа.

Шаг 1 — выходим и заходим заново FRONTEND

Platform Admin

Run every store on your marketplace from one place.
Tarants, catalog, commissions, delivery network and reporting — unified across every store on the platform.

© 2020 Dynamic Coding with Aard

Sign In

Access the platform admin console

Work Email

you@platform.com

Password

Forgot Password?

Sign In

Two-factor disabled

Network

5 / 93 requests | 5.5 kB / 207 kB transferred | 0.8 kB / 13,758 kB resources

Name	Status	Type	Initiator	Size	Time
me	304	xhr	auth.js22	1.0 kB	438 ms
me	304	xhr	auth.js22	1.0 kB	896 ms
2fa-generate-otp	200	xhr	auth.js22	1.1 kB	517 s
2fa-verify-enable	200	xhr	auth.js22	1.1 kB	938 ms
logout	200	xhr	auth.js22	1.1 kB	464 ms

Кнопка «Log out» в боковом меню - сессия на сервере отозвана, локальные токены стёрты.

Platform Admin

Run every store on your marketplace from one place.
Tarants, catalog, commissions, delivery network and reporting — unified across every store on the platform.

© 2020 Dynamic Coding with Aard

Sign In

Access the platform admin console

Work Email

shiginomaksim@gmail.com

Password

Forgot Password?

Sign In

Network

0 / 72 requests | 0.0 kB / 9.8 kB transferred | 0.0 kB / 13,471 kB resources | Finish 1.0 s | DONE

Name	Status	Type	Initiator	Size	Time
------	--------	------	-----------	------	------

Снова вводим email и пароль.

Шаг 2 — сразу на дашборде FRONTEND BACKEND

Platform

Dashboard

Sellers

Commission & Payouts

Global Catalog

Delivery Network

Customers

Orders

Supports

Marketing

Reports

Users & Permissions

Settings

Shigino Maksim

shiginomaksim@gmail.com

DashboardPage

Network

3 / 82 requests | 4.0 kB / 17.8 kB transferred | 1.2 kB / 13,518 kB resources

Name	Status	Type	Initiator	Size	Time
login	200	xhr	auth.js38	1.7 kB	465 ms
me	200	xhr	auth.js22	1.4 kB	438 ms
me	304	xhr	auth.js22	1.0 kB	874 ms

Один запрос login (200) - и сразу настоящие токены доступа, без промежуточного шага. Дальше те подгружает профиль.

Это тот же самый метод `AuthService.login`, что и в разделе 1, просто условие `if (user.twoFactorEnabled)` в этот раз не срабатывает — и код сразу переходит к выдаче токенов:

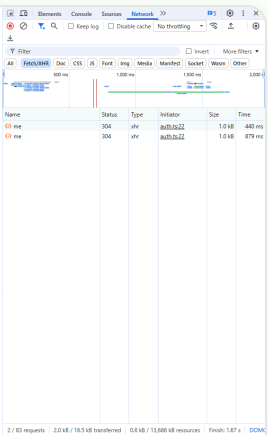
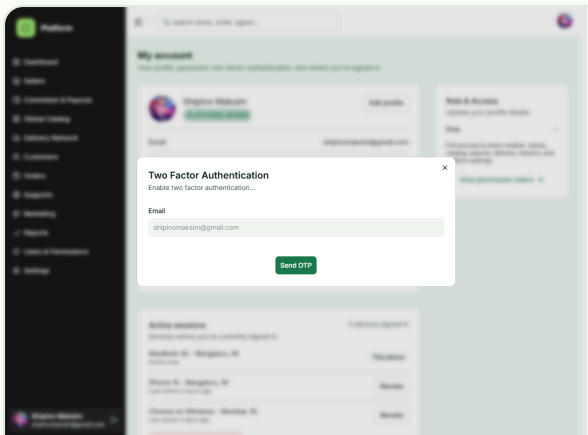
```
const tokens = await this.issueTokenPair(user, context);  
return { ...tokens, userType: user.userType };
```

Что именно выдаётся

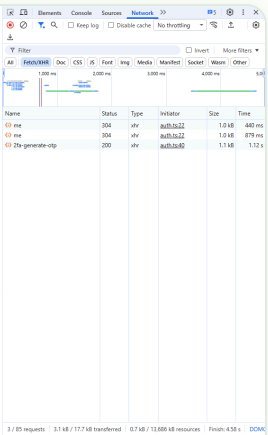
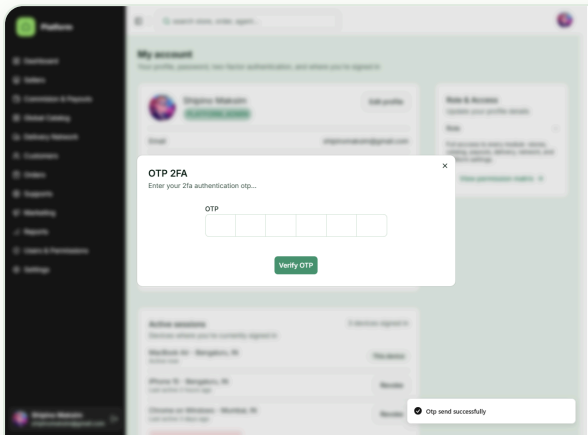
Два токена: короткоживущий access-токен (по нему сервер узнаёт "кто вы" при каждом запросе, живёт 15 минут) и долгоживущий refresh-токен (нужен только для того, чтобы незаметно получить новый access-токен, когда старый истечёт). Плюс в базе создаётся запись о сессии - устройство, IP, время входа - её видно в разделе «Active sessions».

4 Повторное включение 2FA

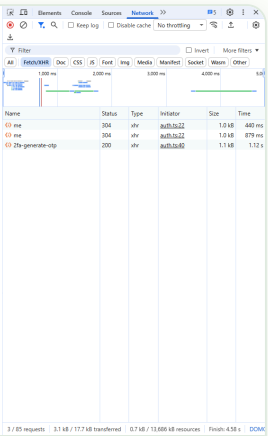
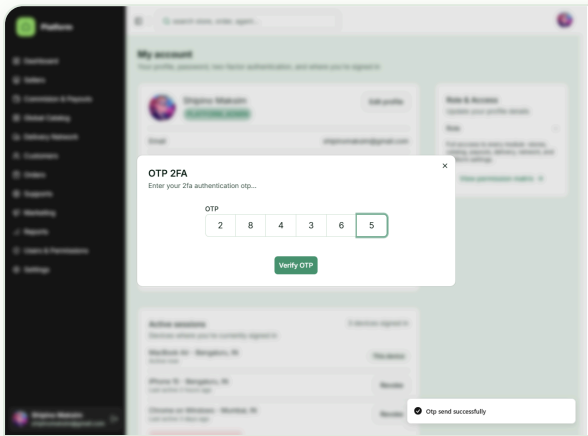
Возвращаем защиту обратно — ровно тот же переключатель, что и в разделе 2, только в другую сторону.



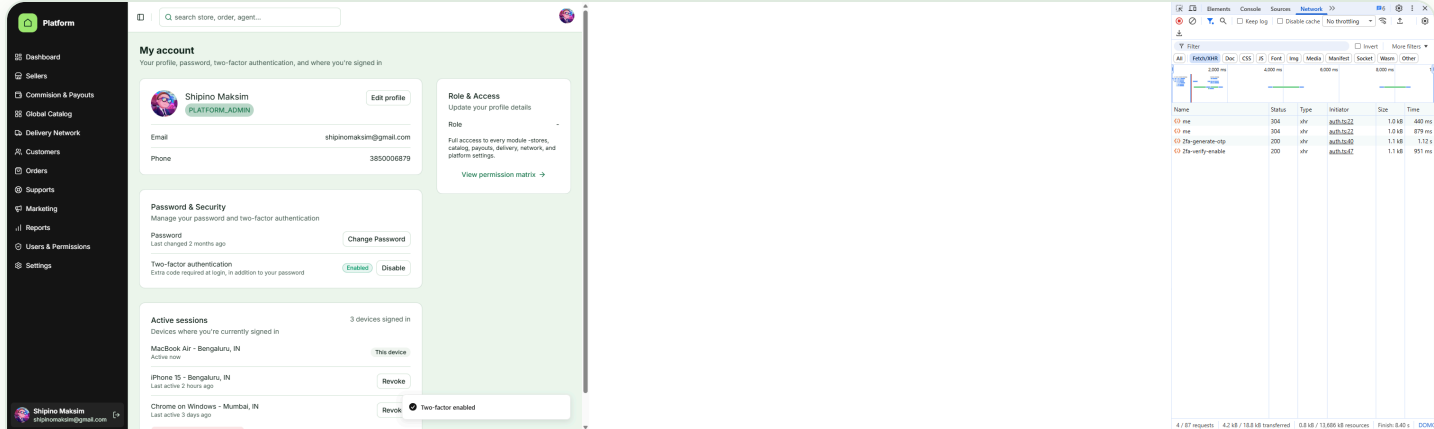
Кнопка снова называется «Enable» - 2FA сейчас выключена.



Новый код сгенерирован и отправлен - запрос 2fa-generate-otp (200).



Код введён.



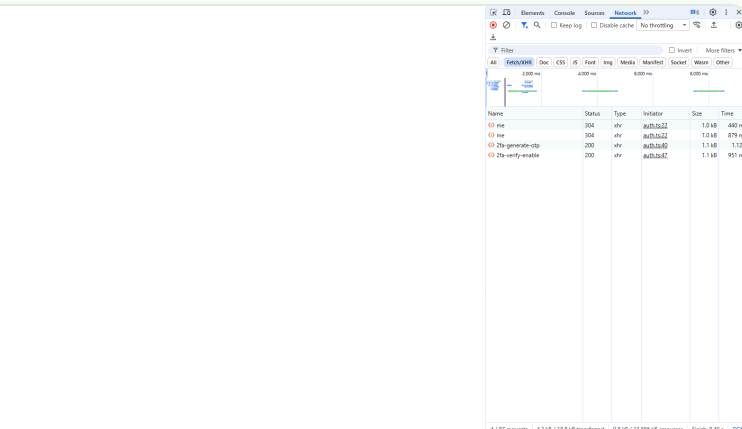
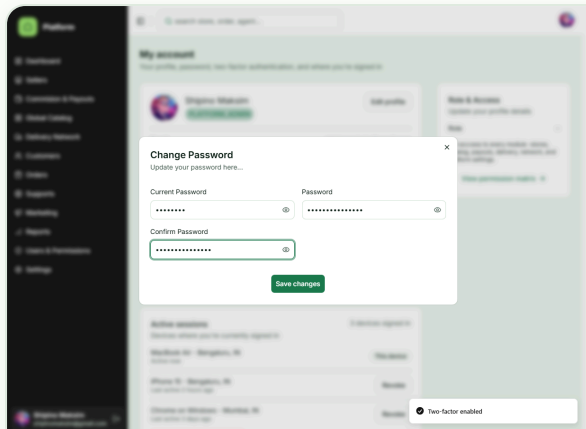
Запрос 2fa-verify-enable (200) - 2FA снова включена.

Итог

Код внутри - буквально тот же самый метод `twoFactorVerifyEnable` из раздела 2. Он не спрашивает "включить" или "выключить" - он просто ставит противоположное значение тому, что было. Именно поэтому фронтенд сам решает, какую надпись показать на кнопке ("Enable" или "Disable"), глядя на текущее состояние.

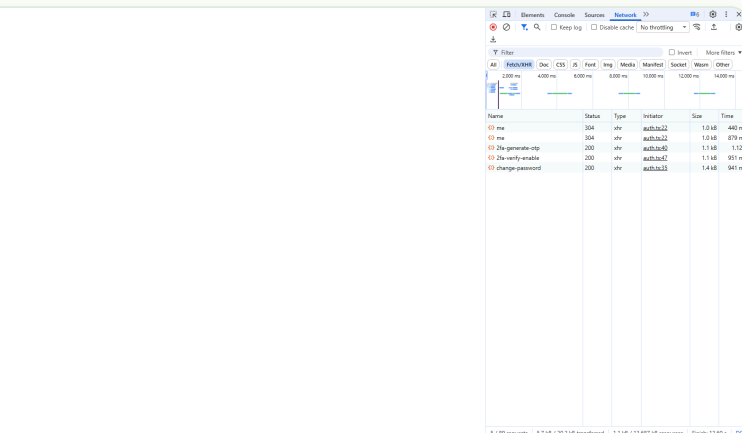
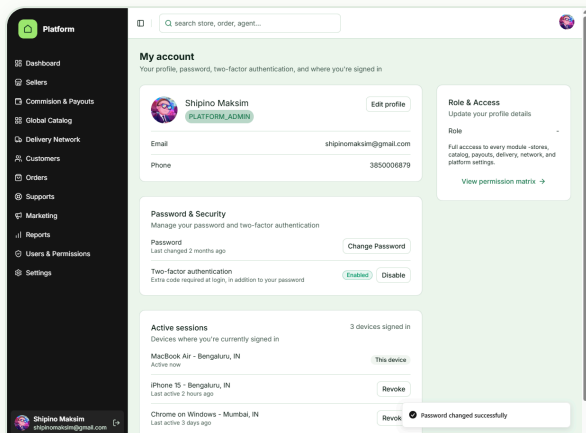
Отдельная форма, отдельный адрес на сервере, ничего общего с редактированием профиля — и это осознанное решение, разберём почему.

Шаг 1 — заполняем форму **FRONTEND**

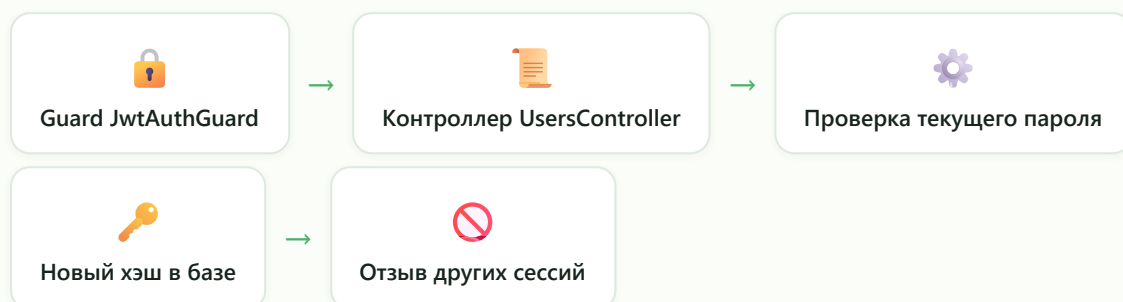


Текущий пароль, новый пароль и его подтверждение.

Шаг 2 — пароль изменён **FRONTEND** **BACKEND**



Запрос change-password (200) - и уведомление «Password changed successfully».



TYPESCRIPT

```
// backend/src/users/users.service.ts
async changePassword(userId: string, dto: ChangePasswordDto, currentSessionId?: string) {
  const existingUser = await this.prisma.user.findUnique({ where: { id: userId } });
```

```

const currentPasswordMatches = await bcrypt.compare(
  dto.currentPassword,
  existingUser.passwordHash,
);
if (!currentPasswordMatches) {
  throw new UnauthorizedException('Current password is incorrect');
}

const passwordHash = await bcrypt.hash(dto.password, 10);
const user = await this.prisma.user.update({
  where: { id: userId },
  data: { passwordHash },
});

// Если пароль скомпрометирован и им уже кто-то пользуется на другом
// устройстве – эта смена пароля выкидывает его оттуда. Своё же
// текущее устройство (с которого только что сменили пароль) не трогаем.
await this.prisma.userSession.updateMany({
  where: { userId, revokedAt: null, id: { not: currentSessionId } },
  data: { revokedAt: new Date() },
});

return { user, message: 'Password changed successfully', success: true };
}

```

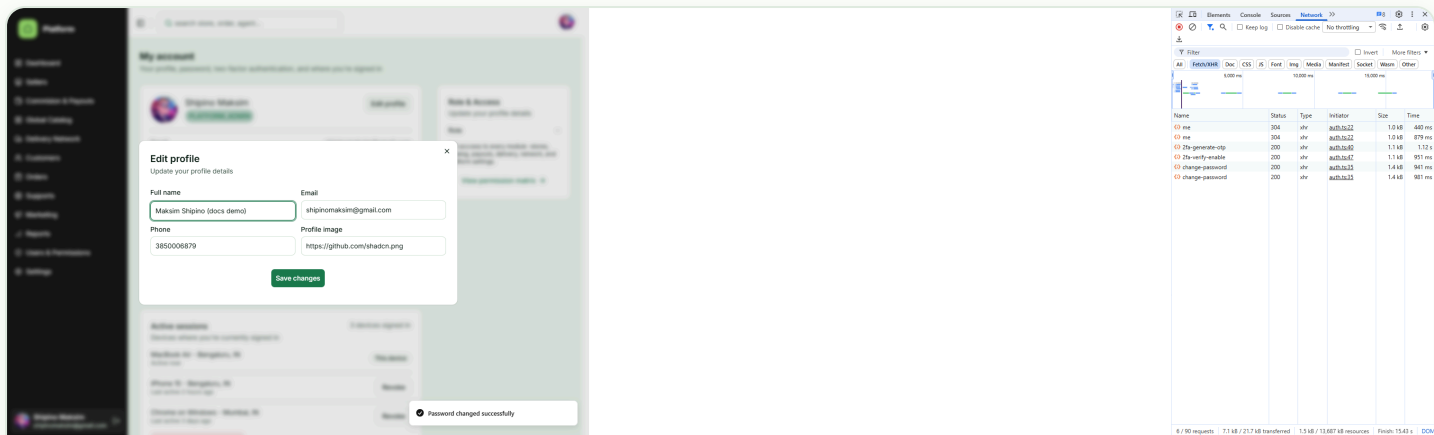
Зачем спрашивать текущий пароль, если человек и так уже вошёл в аккаунт?

Потому что "вошёл в аккаунт" и "знает пароль" - разные вещи. Например, если кто-то украл активную сессию (токен) через уязвимость в браузере или получил доступ к разблокированному телефону, он сможет открыть настройки, но не будет знать текущий пароль. Требование ввести его ещё раз не даёт злоумышленнику тихо перехватить аккаунт себе, сменив пароль без ведома владельца.

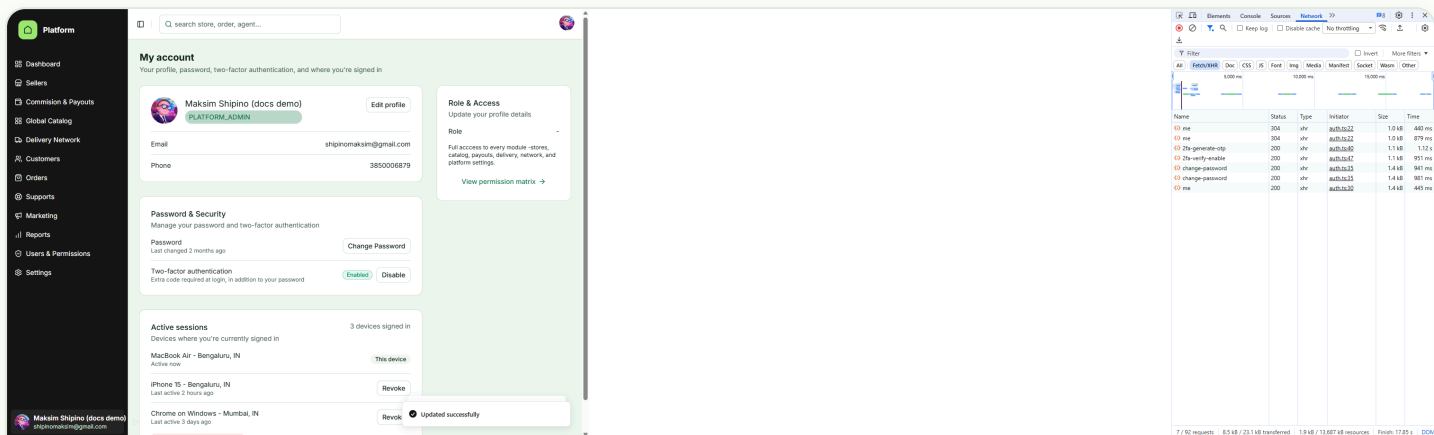
Почему это отдельная форма, а не часть "Редактировать профиль"?

У формы редактирования профиля (имя, e-mail, телефон) на сервере нет ни единого поля для пароля - оно физически отсутствует в структуре допустимых данных этого запроса. Значит, даже теоретически через эту форму пароль сменить нельзя. Единственный путь изменить пароль - через отдельный адрес, который дополнительно требует текущий пароль. Это уменьшает количество мест, где можно случайно (или намеренно) утечь эту возможность.

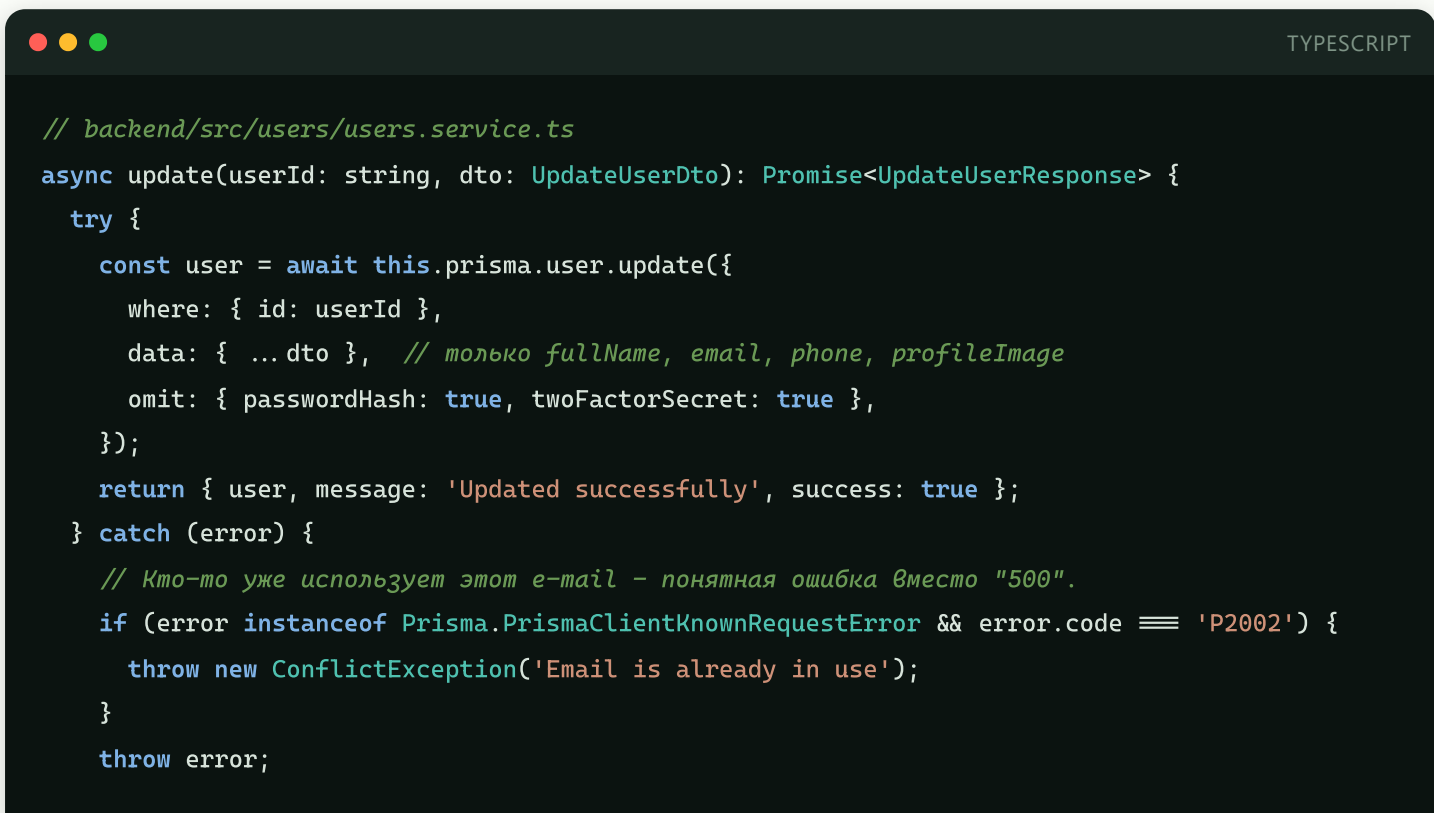
Имя, e-mail, телефон, фото — самая обычная форма «обновить данные».



Имя изменено, кнопка «Save changes» стала активной (до первого изменения она заблокирована).



Запрос PUT /users/me (200) - профиль обновлён, уведомление «Updated successfully».



```
}  
}  
}
```

Почему кнопка «Save changes» была неактивна до правки?

Фронтенд сравнивает текущие значения полей с исходными и блокирует сохранение, пока нет ни одного реального изменения - это просто избавляет от лишнего запроса на сервер, когда сохранять на самом деле нечего.

А что если ввести поле, которого нет в форме?

Не получится даже искусственно - на входе в сервер стоит общий фильтр (ValidationPipe с whitelist), который вообще отбрасывает любые поля, не описанные в разрешённом списке для этого запроса, и с настройкой forbidNonWhitelisted вернёт ошибку 400, если такое поле обнаружено.

7 Что защищает аккаунт: итоги

Коротко, без кода - что происходит "под капотом" всегда, вне зависимости от конкретного экрана.

Защита	Что это значит простыми словами
Пароли и коды не хранятся в открытом виде	В базе лежит только необратимый «отпечаток» (хэш). Даже у разработчиков нет способа посмотреть исходный пароль или код.
Одинаковый ответ на неверный email и неверный пароль	Сообщение об ошибке при входе не выдаёт, зарегистрирован ли вообще такой email в системе.
Ограничение попыток ввода кода	После 5 неверных попыток код перестаёт приниматься - перебором его не подобрать.
Ограничение частоты запросов (Rate limiting)	Чувствительные адреса (вход, коды 2FA) не дадут постучаться чаще нескольких раз в минуту с одного места.
Разные секреты для разных токенов	Ключ доступа, ключ обновления сессии и ключ для кода 2FA подписаны разными секретами - утечка одного не подделывает остальные.
Сессии можно отозвать	Даже если токен технически ещё не истёк, запись о сессии в базе можно пометить «отозвана» - и доступ пропадёт мгновенно (например, при выходе или смене пароля).
Смена пароля требует текущий пароль	Захват активной сессии сам по себе не даёт возможности тихо увести аккаунт себе.
Строгая проверка входных данных	Любое поле, не предусмотренное конкретной формой, сервер отклоняет ещё до того, как оно попадёт в бизнес-логику.

Документ сгенерирован автоматически на основе реального прогона приложения (Playwright) поверх фактического кода бэкенда и фронтенда платформы. Дата: 18 сентября 2026 г